

Análise Estatística de Métricas de Código-Fonte do Framework Neo4j

Igor Mariano Lopes Rodrigues¹, Filipe Oliveira de Saldanha da Gama¹

IBMEC – Curso de Graduação em Engenharia de Software
{202407095992, 202303161085}@alunos.ibmec.edu.br

Abstract.

This paper presents a statistical analysis of source code metrics from the Neo4j framework using the GitHub Bug Dataset v1.1, covering class-level (7,917 instances) and file-level (4,261 instances) granularities. Descriptive statistics, normality tests (Shapiro-Wilk and Lilliefors), Pearson and Spearman correlations, VIF analysis, and cost-sensitive logistic regression with cross-validation were applied. Strong correlations among size and complexity metrics were found (e.g., LOC×LLOC: $r = 0.99$). A severe class imbalance ($< 0.15\%$ positive) fundamentally limits predictive modeling—with only 8 and 6 bug instances, the Events Per Variable ratio is far below the recommended minimum of 10.

Resumo.

Este artigo apresenta uma análise estatística de métricas de código-fonte do framework Neo4j, utilizando o GitHub Bug Dataset v1.1, nos níveis classe (7.917 instâncias) e arquivo (4.261 instâncias). Foram aplicados estatística descritiva, testes de normalidade (Shapiro-Wilk e Lilliefors), correlações de Pearson e Spearman, análise VIF e regressão logística *cost-sensitive* com validação cruzada. Correlações fortes foram encontradas entre métricas de tamanho e complexidade (e.g., LOC×LLOC: $r = 0,99$). O desbalanceamento severo ($< 0,15\%$ positivos) limita fundamentalmente a modelagem preditiva—com apenas 8 e 6 instâncias com bug, a razão EPV está muito abaixo do mínimo de 10.

1 Introdução

A análise quantitativa de métricas de código-fonte constitui abordagem fundamental na Engenharia de Software para compreender qualidade, complexidade e propensão a defeitos [9]. Métricas orientadas a objetos, como as de Chidamber e Kemerer [2], permitem avaliar características estruturais de projetos Java e são amplamente utilizadas em predição de falhas [3, 1].

Este trabalho aplica técnicas de estatística descritiva e inferencial sobre datasets de métricas do framework Neo4j, extraídos do GitHub Bug Dataset v1.1 [13]. A análise contempla: (i) medidas de tendência central e dispersão; (ii) gráficos e detecção de outliers; (iii) testes de normalidade; (iv) correlação (Pearson e Spearman) e multicolinearidade (VIF); e (v) regressão logística *cost-sensitive* validada por *cross-validation*. O repositório está em: <https://github.com/igormariano/AP1-Projeto-ES-Neo4j>.

2 Metodologia

2.1 Fonte de Dados

Os datasets foram obtidos do GitHub Bug Dataset v1.1 [13]: **Class-level** com 7.917 instâncias e 113 atributos (complexidade, acoplamento, coesão, herança, tamanho, documentação, warnings) e **File-level** com 4.261 instâncias e 12 atributos (McCC, LLOC, histórico de modificações). Nenhum dataset apresentou valores ausentes.

2.2 Seleção de Variáveis

Dos 113 atributos do Class-level, selecionaram-se 19 variáveis numéricas (Tabela 1) por relevância estatística e ausência de redundância. Excluíram-se identificadores textuais, metadados de localização, métricas totais redundantes (prefixo T), categorias individuais de regras PMD e variáveis com variância zero. No File-level, CLOC foi excluída por ser constante, mantendo-se 7 variáveis.

Tabela 1: Variáveis selecionadas para análise (Class-level)

Variável	Categoria	Descrição
WMC, NL, NLE	Complexidade	Complexidade ponderada, aninhamento
CBO, RFC	Acoplamento	Acoplamento entre classes, resposta
LCOM5	Coesão	Falta de coesão dos métodos
DIT, NOC	Herança	Profundidade, número de filhos
LOC, LLOC, NOS, NM, NA	Tamanho	Linhas, statements, métodos, atributos
CC	Clones	Cobertura de clones
CD, CLOC	Documentação	Densidade e linhas de comentário
WarningInfo, WarningMajor	Qualidade	Warnings informativos e majoritários
N. de bugs	Alvo	Número de defeitos

2.3 Ferramentas Utilizadas

Exploração inicial em Python 3.12 (pandas, numpy). Análise definitiva em R 4.5.2 [10]: ggplot2, corrplot, PerformanceAnalytics, car, nortest, pROC, PRROC e caret.

3 Resultados e Discussão

3.1 Estatísticas Descritivas

A Tabela 2 sintetiza as medidas de tendência central e dispersão para as principais variáveis do Class-level.

Tabela 2: Estatísticas descritivas — Class-level						
Var.	Média	Mediana	Moda	Amp.	DP	CV
WMC	7,39	3	1	325	13,90	1,88
CBO	5,64	3	2	161	7,45	1,32
RFC	9,84	5	1	293	16,01	1,63
LOC	59,70	25	7	3.207	115,59	1,94
LLOC	49,18	21	8	2.588	92,05	1,87
NOS	18,77	5	1	1.411	47,10	2,51
NM	8,61	4	1	161	13,56	1,57
DIT	0,98	1	1	6	0,99	1,01
CC	0,12	0	0	1	0,29	2,42

Em todas as métricas, a média é substancialmente superior à mediana, indicando distribuições fortemente assimétricas à direita. LOC apresenta $CV = 1,94$, confirmando alta dispersão. A moda igual a 1 para WMC, RFC, NOS e NM indica que a maioria das classes possui complexidade mínima. DIT apresenta baixa variabilidade (amplitude = 6, $\sigma = 0,99$), refletindo hierarquias rasas.

Para o File-level, McCC apresenta média de 6,06 e mediana de 2,00; LLOC tem média de 102,74 e mediana de 55,00. “Number of developer commits” apresenta variância de 4.106.050,59, decorrente da grande discrepância entre desenvolvedores.

3.2 Boxplots e Outliers

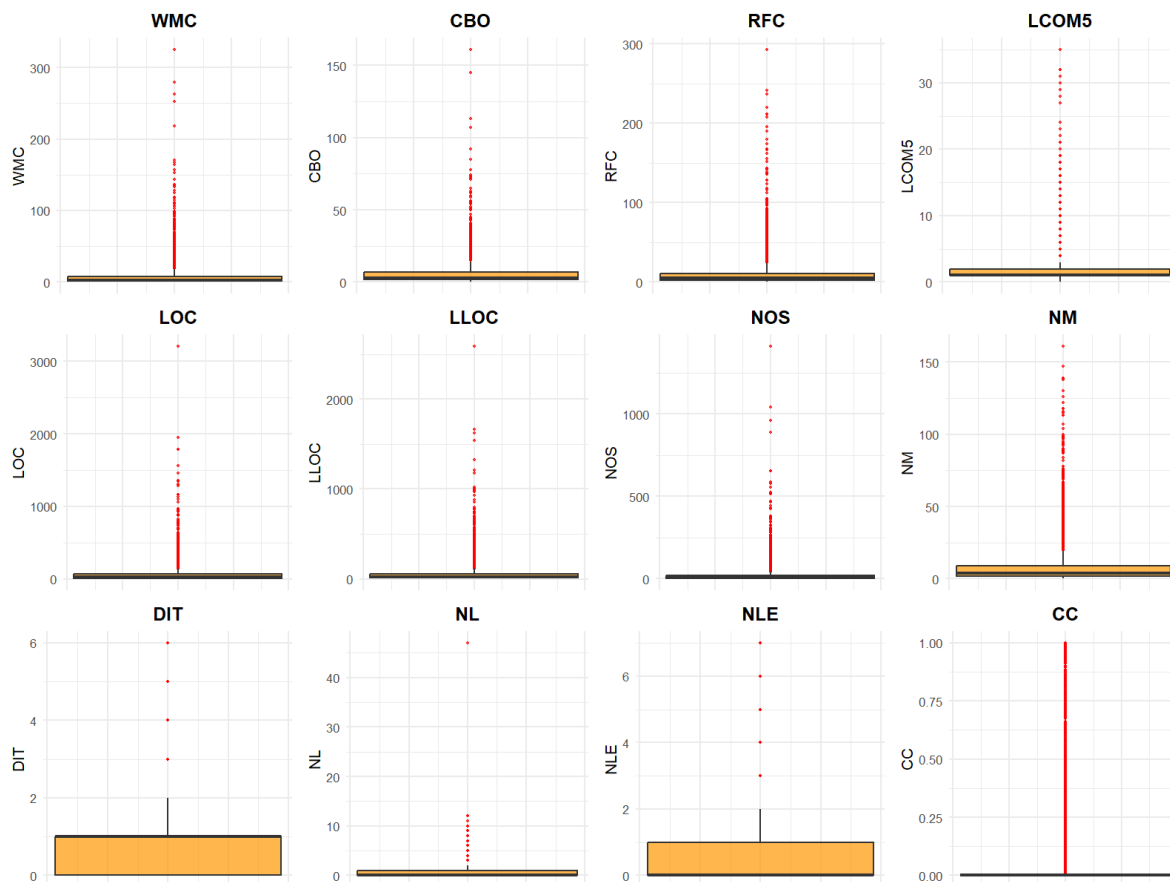


Figura 1: Boxplots das principais métricas — Class-level

A Figura 1 evidencia a presença massiva de outliers em todas as métricas. A Tabela 3 resume a detecção pelo método IQR. A proporção varia entre 5,86% (NA) e 22,48% (CC). Para métricas de tamanho (LOC, LLOC, NOS), $\approx 10\%$ são outliers, consistente com a regra de Pareto [8]. Os outliers não foram removidos por representarem classes legitimamente complexas. O File-level apresenta padrão análogo (McCC: 10,91%; LLOC: 7,89% de outliers).

Tabela 3: Outliers pelo método IQR — Class-level (seleção)

Variável	Outliers	%	Limite Sup.
WMC	706	8,92	18,50
CBO	644	8,13	14,50
RFC	759	9,59	24,50
LOC	799	10,09	140,00
NOS	840	10,61	42,00
NM	866	10,94	19,50
CC	1.780	22,48	0,00

3.3 Histogramas e Distribuições

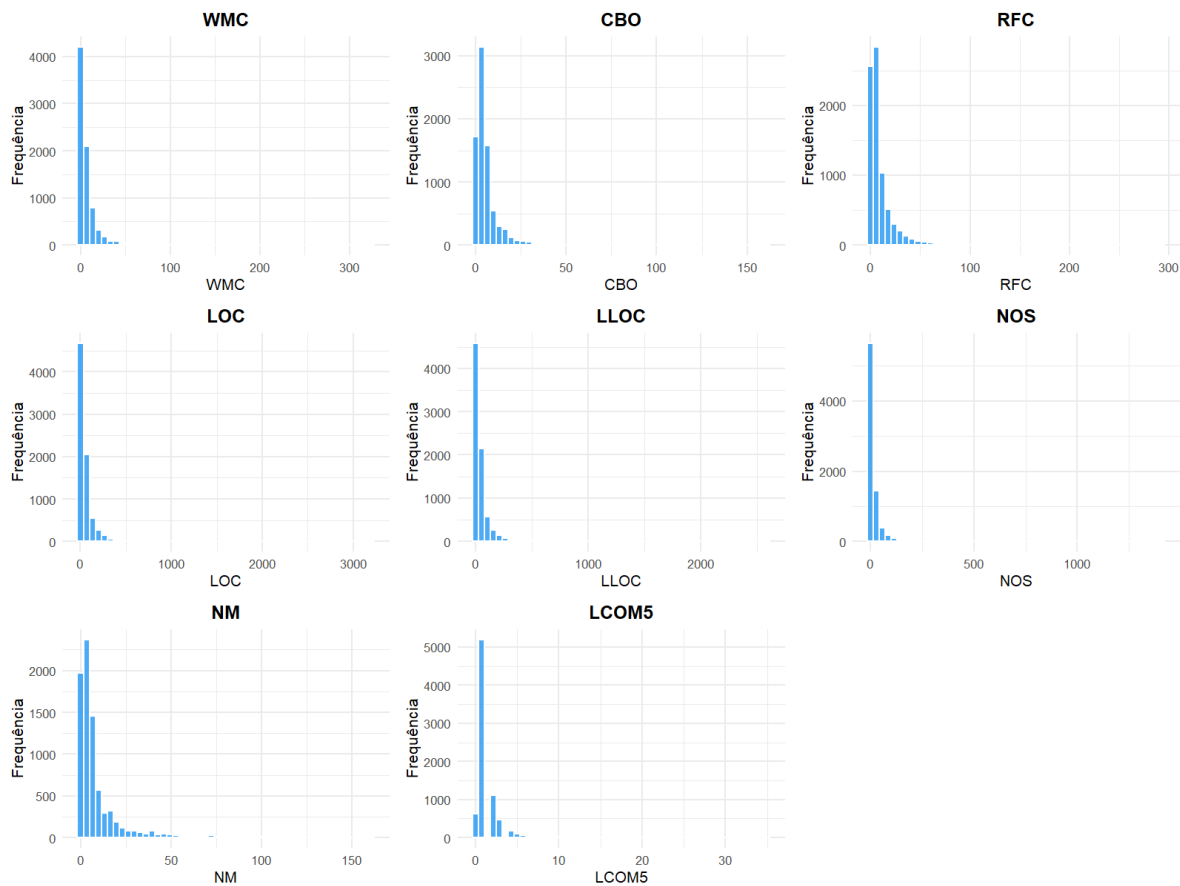


Figura 2: Histogramas das métricas Class-level

Os histogramas (Figura 2) confirmam distribuições fortemente assimétricas à direita, com concentração massiva em valores baixos e caudas longas, comportamento esperado em métricas de software [11]. A distribuição de bugs é extremamente desbalanceada: apenas 8 classes (0,10%) e 6 arquivos (0,14%) apresentam bugs registrados.

3.4 Testes de Normalidade

Para todas as variáveis, os testes de Shapiro-Wilk [12] e Lilliefors [5] rejeitaram a hipótese nula de normalidade ($p < 10^{-50}$). A Tabela 4 apresenta estatísticas para variáveis representativas. Adotou-se Lilliefors em lugar do teste K-S com parâmetros estimados, que é anticonservativo [5]. Para Shapiro-Wilk (limite de 5.000 obs.), utilizou-se amostra com seed=42.

Tabela 4: Testes de normalidade — Class-level

Var.	SW	SW p	Lill. D	Lill. p
WMC	0,421	$< 10^{-83}$	0,308	$< 2,2 \times 10^{-16}$
CBO	0,587	$< 10^{-76}$	0,225	$< 2,2 \times 10^{-16}$
LOC	0,424	$< 10^{-83}$	0,311	$< 2,2 \times 10^{-16}$
DIT	0,827	$< 10^{-59}$	0,252	$< 2,2 \times 10^{-16}$

A rejeição universal de normalidade justifica o uso de Spearman e implica cautela com métodos que assumem normalidade.

3.5 Análise de Correlação

Dado que todas as variáveis violam normalidade, complementou-se Pearson com Spearman (baseada em postos), mais robusta a outliers.

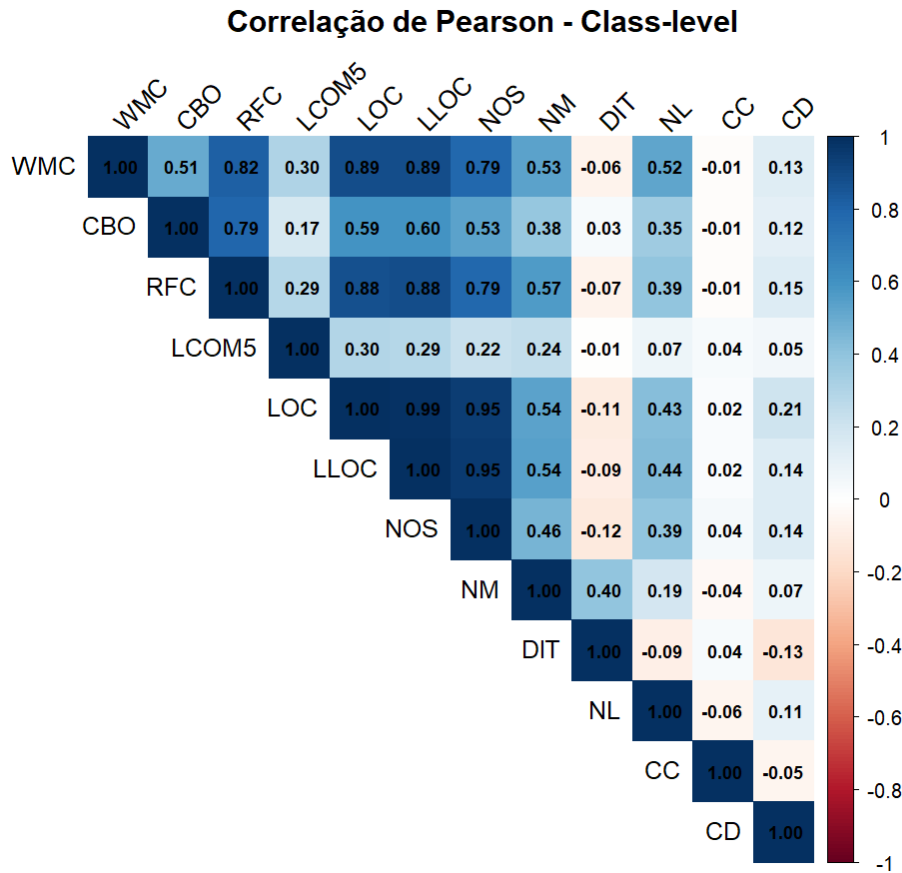


Figura 3: Heatmap de correlação de Pearson — Class-level

Correlações fortes ($|r| \geq 0,7$, Pearson): LOC×LLOC: $r = 0,99$; LOC×NOS: $r = 0,95$; WMC×LOC: $r = 0,89$; WMC×RFC: $r = 0,82$; CBO×RFC: $r = 0,80$. **Moderadas** ($0,4 \leq |r| < 0,7$): WMC×CBO: $r = 0,51$; CBO×LOC: $r = 0,59$. As correlações de Spearman apresen-

tam padrão semelhante com magnitudes geralmente mais elevadas (e.g., LOC×LLOC: $\rho = 0,99$; NOS×NL: $\rho = 0,72$). A convergência reforça a robustez dos achados.

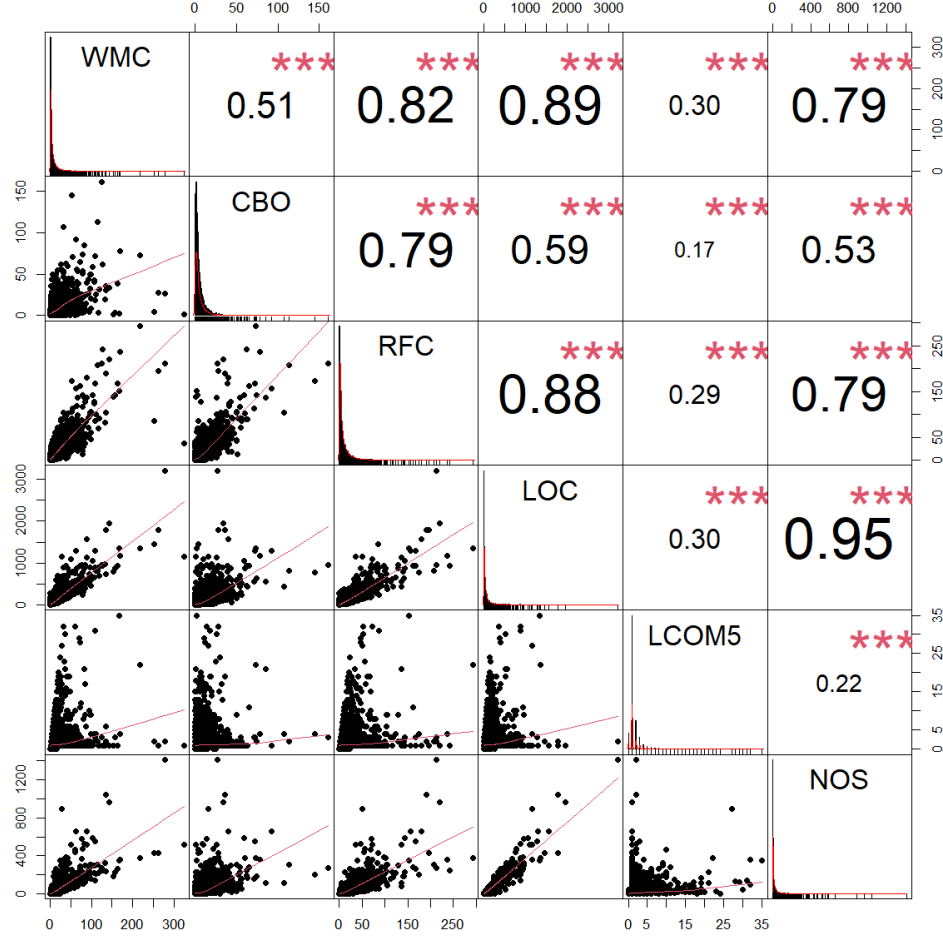


Figura 4: chart.Correlation — métricas principais (Class-level)

A Figura 4 combina histogramas, dispersão e coeficientes de correlação. No File-level, as correlações mais fortes são McCC×LLOC ($r = 0,72$) e “N. previous modifications”×“N. committers” ($r = 0,77$; Spearman: $\rho = 0,90$).

3.6 Análise de Multicolinearidade (VIF)

Calculou-se o VIF para quantificar multicolinearidade [7]. No Class-level, RFC (VIF = 20,18), WMC (15,17) e LOC (10,98) excederam o limiar de 10. Após remoção iterativa de RFC, os VIF finais ficaram abaixo de 10 (WMC: 8,75; LOC: 9,16). No File-level, todos os VIF foram aceitáveis ($\leq 5,58$).

3.7 Regressão Logística

Optou-se por regressão logística binária com pesos *cost-sensitive* inversamente proporcionais à frequência de cada classe, evitando classificação trivial como classe majoritária.

3.7.1 Modelo Class-level

O modelo utilizou preditores remanescentes após remoção de $VIF > 10$ (WMC, CBO, LOC, LCOM5, NL, DIT, CC). Os coeficientes apresentaram magnitude extrema (ordem 10^{12} – 10^{15}), indicando que o algoritmo *não convergiu* — separação completa dos dados, sintoma do desbalanceamento extremo (8 positivos em 7.917, $EPV \approx 1$). O Pseudo R^2 (McFadden) foi $-10,76$ (modelo pior que o nulo). Na escala de McFadden, 0,2–0,4 indica ajuste excelente [6].

3.7.2 Modelo File-level

Os preditores foram McCC, LLOC, N. previous modifications, N. previous fixes, N. committers e N. developer commits. O Pseudo R^2 (McFadden) foi 0,59 (excelente), porém o algoritmo também não convergiu. Apenas LLOC não foi significativo ($p = 0,30$); os demais resultados devem ser interpretados com cautela dado o número ínfimo de positivos (6 instâncias).

3.7.3 Métricas de Avaliação e Validação Cruzada

Tabela 5: Métricas de avaliação: regressão e validação cruzada (10-fold)

Métrica	Treino completo		10-fold CV	
	Class	File	Class	File
AUC-ROC	0,674	0,968	0,660	0,805
AUC-PR	0,007	0,020	—	—
Sensitivity	0,375	1,000	0,625	0,833
Precision	0,014	0,024	0,002	0,007
F1-score	0,027	0,047	0,004	0,014
Bal. Acc.	0,674	0,972	0,664	0,836
MCC	0,069	0,151	0,023	0,068

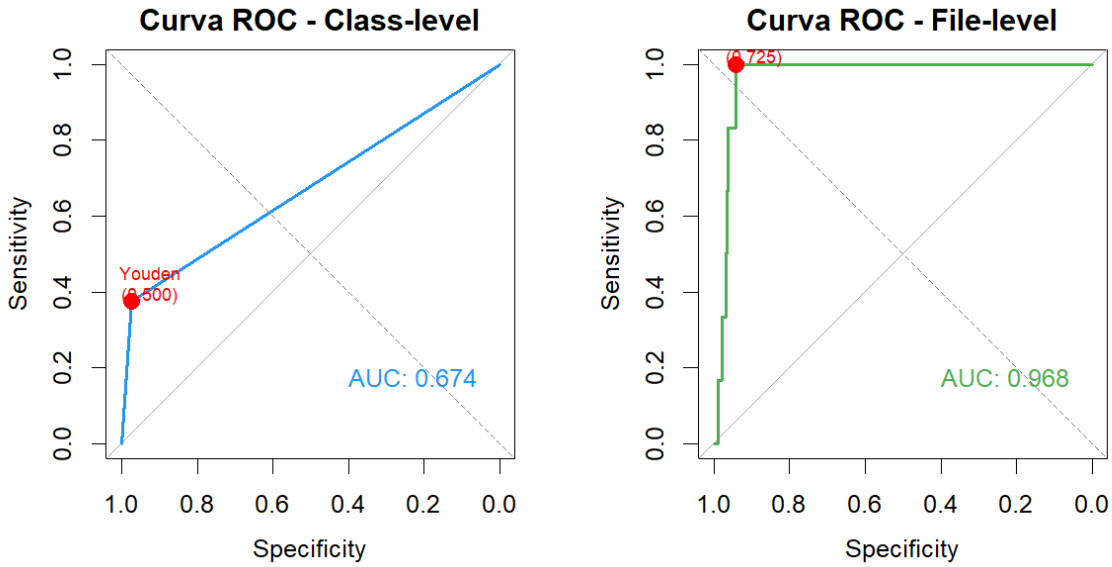


Figura 5: Curvas ROC com Youden Index para Class e File-level

A Tabela 5 consolida as métricas do treino completo e da validação cruzada estratificada com 10 folds. A degradação na CV confirma overfitting, especialmente no File-level (AUC: 0,97 $\rightarrow$ 0,81). Os valores de MCC próximos de zero (0,023 e 0,068) indicam poder preditivo praticamente nulo. A “acurácia” de 97–99% é enganosa: um classificador trivial que sempre prediz “sem bug” obtém desempenho similar. As curvas Precision-Recall (não apresentadas por limitação de espaço) confirmam AUC-PR $\approx$ 0, reforçando a inadequação dos dados para predição.

3.8 Discussão

As métricas do Neo4j seguem distribuições fortemente assimétricas, consistente com a literatura [11, 3]. As correlações fortes entre LOC, LLOC, NOS, WMC e RFC indicam dimensões sobrepostas de tamanho e complexidade, confirmadas pela análise VIF. A correlação extremamente fraca entre métricas e bugs (máximo $r = 0,06$) contrasta com estudos anteriores [1, 3], explicável pelo desbalanceamento extremo e possível subreporte de defeitos.

Quanto às métricas CK: DIT e NOC possuem variância quase zero no Neo4j (design “flat”), tornando-as pouco discriminativas. LCOM5 possui múltiplas definições na literatura, dificultando comparações entre estudos.

3.9 Ameaças à Validade

Interna: (i) Desbalanceamento extremo da variável alvo (8 e 6 positivos; $EPV \approx 1$, muito abaixo do mínimo de 10 [4]); (ii) Teste K-S original com parâmetros estimados foi anticonservativo — corrigido com Lilliefors; (iii) Multicolinearidade severa (LOC $\times$ LLOC: $r = 0,99$) — corrigida via VIF; (iv) Modelo avaliado no conjunto de treino — corrigido com 10-fold CV.

Externa: Resultados específicos ao Neo4j; dataset de versão única; possível viés de seleção do GitHub Bug Dataset.

Construto: Métricas CK são proxies imperfeitos de qualidade; “Number of bugs” depende da completude do rastreamento; DIT e NOC com variância quase zero são irrelevantes neste contexto.

4 Conclusão

Apresentou-se uma análise estatística de métricas do Neo4j (GitHub Bug Dataset v1.1) com testes de normalidade corrigidos (Lilliefors), correlação de Pearson e Spearman, análise VIF, regressão *cost-sensitive* e validação cruzada. Todas as métricas seguem distribuições assimétricas com correlações fortes entre tamanho e complexidade (LOC×LLOC: $r = 0,99$). O principal achado é a **inadequação do dataset para predição de defeitos**: com EPV ≈ 1 , os modelos são não confiáveis e a “alta acurácia” é enganosa.

Como trabalhos futuros: (i) combinar múltiplas versões do Neo4j para aumentar instâncias positivas; (ii) aplicar *Random Forest* ou *gradient boosting*; (iii) explorar *cross-project defect prediction*.

Referências

- [1] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering*, 22(10):751–761, 1996.
- [2] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [3] Tibor Gyimóthy, Rudolf Ferenc, and István Siket. Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software Engineering*, 31(10):897–910, 2005.
- [4] David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, 3rd edition, 2013.
- [5] Hubert W. Lilliefors. On the Kolmogorov-Smirnov test for normality with mean and variance unknown. *Journal of the American Statistical Association*, 62(318):399–402, 1967.
- [6] Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974.
- [7] Douglas C. Montgomery and George C. Runger. *Applied Statistics and Probability for Engineers*. John Wiley & Sons, 5th edition, 2012.
- [8] Nachiappan Nagappan, Thomas Ball, and Andreas Zeller. Mining metrics to predict component failures. *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, pages 452–461, 2006.
- [9] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill Education, 8th edition, 2014.

- [10] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.
- [11] Danijel Radjenović, Marjan Heriko, Richard Torkar, and Aleš Živkovič. Software fault prediction metrics: A systematic literature review. *Information and Software Technology*, 55(8):1397–1418, 2013.
- [12] Samuel S. Shapiro and Martin B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3/4):591–611, 1965.
- [13] Zoltán Tóth, Péter Gyimési, and Rudolf Ferenc. A public bug database of GitHub projects and its application in bug prediction. In *Proceedings of the International Conference on Computational Science and Its Applications (ICCSA)*, pages 625–638. Springer, 2016.